

Lab Work for LAB 5: Django Forms and User Input with Bootstrap

Objective

By the end of this lab, students will:

1. Understand Django forms and how to collect user input.
 2. Implement a form for student enrollment in courses.
 3. Use Bootstrap to style the form for a professional UI.
 4. Validate and process form submissions.
-

Step 1: Create a Django Form for Course Enrollment

1. **Task:** Create a forms.py file in the courses app (if it does not exist).
 - Add the following code:

```
from django import forms
```

```
from .models import Student, Course
```

```
class CourseEnrollmentForm(forms.Form):
```

```
    student_email = forms.EmailField(label="Student Email", widget  
                                     = forms.EmailInput(attrs={'class': 'form-control'}))
```

```
    course = forms.ModelChoiceField(queryset=Course.objects.all(), label  
                                    = "Select Course", widget=forms.Select(attrs  
                                    = {'class': 'form-control'}))
```

2. **Why This Matters:** Forms allow user input to be collected in a structured way.
 3. **Bootstrap Integration:** Notice how we use attrs={'class': 'form-control'} to apply Bootstrap styles.
-

Step 2: Create a View for Form Handling

1. **Task:** Add a new view in views.py to handle the form submission.
 - Open views.py and add the following:

```
from django.shortcuts import render, redirect
```

```
from .forms import CourseEnrollmentForm
```

```
from .models import Student
```

```
def enroll_student(request):
```

```

if request.method == 'POST':
    form = CourseEnrollmentForm(request.POST)
    if form.is_valid():
        student_email = form.cleaned_data['student_email']
        course = form.cleaned_data['course']

        # Get or create the student
        student, created = Student.objects.get_or_create(email = student_email)
        student.enrolled_courses.add(course)

    return render(request, 'courses
                    /enrollment_success.html', {'student': student, 'course': course})
else:
    form = CourseEnrollmentForm()

return render(request, 'courses/enroll_student.html', {'form': form})

```

2. Why This Matters:

- The view handles **GET** requests (to display the form) and **POST** requests (to process submissions).
- `get_or_create` ensures a student is registered before enrolling in a course.

Step 3: Define URL Patterns

1. **Task:** Add a new URL pattern in `urls.py`:

```

from django.urls import path
from . import views

```

```

urlpatterns = [
    path('', views.course_list, name = 'course_list'),
    path('<int: course_id >/', views.course_detail, name = 'course_detail'),
    path('enroll/', views.enroll_student, name = 'enroll_student'),
]

```

2. **Why This Matters:** This allows students to access the enrollment form via `/enroll/`.

Step 4: Create Bootstrap-Styled Templates

1. Enrollment Form Page

1. **Task:** Create enroll_student.html inside templates/courses/:

```
{% extends "courses/base.html" %}

{% block title %}Enroll in a Course{% endblock %}

{% block content %}

<div class = "container" >

  <h1 class = "text - center" > Enroll in a Course </h1 >

  <div class = "row justify - content - center" >

    <div class = "col - md - 6" >

      <form method = "post" class = "p - 4 border rounded bg - white shadow - sm" >

        {% csrf_token %}

        <div class = "mb - 3" >

          <label for = "id_student_email" class = "form - label" > Student Email

            </label >

          {{ form.student_email }}

        </div >

        <div class = "mb - 3" >

          <label for = "id_course" class = "form - label" > Select Course </label >

          {{ form.course }}

        </div >

        <button type = "submit" class = "btn btn - primary w - 100" > Enroll </button

          >

        </form >

      </div >

    </div >

  </div >

{% endblock %}
```

2. Why This Matters:

- Uses Bootstrap for a **modern** and **user-friendly** form.

- Centered form with **shadows and padding** for a professional look.
-

2. Enrollment Success Page

1. **Task:** Create `enrollment_success.html` inside `templates/courses/`:

```
{% extends "courses/base.html" %}
```

```
{% block title %}Enrollment Successful{% endblock %}
```

```
{% block content %}
```

```
<div class = "container" >
```

```
<div class = "alert alert – success text – center" >
```

```
<h2 > Enrollment Successful! </h2 >
```

```
<p > {{ student.name or student.email }} has been enrolled in <strong  
> {{ course.title }} </strong >.</p >
```

```
<a href = "/" class = "btn btn – primary" > Back to Courses </a >
```

```
</div >
```

```
</div >
```

```
{% endblock %}
```

2. **Why This Matters:**

- **Uses Bootstrap's alert component** for a success message.
 - Encourages the user to navigate back to the course list.
-

Step 5: Test the Form Functionality

1. **Task:** Run the Django development server.

```
python manage.py runserver
```

2. **Task:** Access the enrollment form at `http://127.0.0.1:8000/enroll/`.

3. **Task:** Test form submission by:

- Entering an email and selecting a course.
- Clicking "**Enroll**" and verifying the success message.

4. **Task:** Check the database to ensure the student is enrolled in the course.
-

Lab Deliverables

1. **Code:**

- forms.py with CourseEnrollmentForm.
- Enrollment view in views.py.
- Updated urls.py.
- Bootstrap-styled templates (enroll_student.html, enrollment_success.html).

2. Screenshots of:

- The enrollment form page.
- The success page after form submission.

Bonus: Extend the Form

1. **Task:** Modify the form to allow students to enroll with a **name** in addition to an email.

```
class CourseEnrollmentForm(forms.Form):
```

```
    student_name = forms.CharField(label = "Full Name", max_length = 100, widget
                                   = forms.TextInput(attrs = {'class': 'form-control'}))
```

```
    student_email = forms.EmailField(label = "Student Email", widget
                                     = forms.EmailInput(attrs = {'class': 'form-control'}))
```

```
    course = forms.ModelChoiceField(queryset = Course.objects.all(), label
                                    = "Select Course", widget = forms.Select(attrs
                                    = {'class': 'form-control'}))
```

2. **Task:** Modify views.py to save the student's name.

```
def enroll_student(request):
```

```
    if request.method == 'POST':
```

```
        form = CourseEnrollmentForm(request.POST)
```

```
        if form.is_valid():
```

```
            student_name = form.cleaned_data['student_name']
```

```
            student_email = form.cleaned_data['student_email']
```

```
            course = form.cleaned_data['course']
```

```
            student, created = Student.objects.get_or_create(email = student_email)
```

```
            student.name = student_name # Save the name if not already set
```

```
            student.save()
```

```
            student.enrolled_courses.add(course)
```

```
        return render(request, 'courses
                        /enrollment_success.html', {'student': student, 'course': course})
    else:
        form = CourseEnrollmentForm()

    return render(request, 'courses/enroll_student.html', {'form': form})
```

3. **Why This Matters:** This improves the student registration process.
-

Additional Questions for Students

1. How does Django's `forms.Form` differ from `forms.ModelForm`?
2. Why is CSRF protection (`{{ csrf_token }}`) important in forms?
3. How would you modify this form to allow multiple course enrollments at once?